Unit 4

Software Project Management: -

Software Project Management (SPM) is a proper way of planning and leading software projects. It is a part of project management in which software projects are planned, implemented, monitored, and controlled.

Types of Management in SPM

1. Conflict Management

Conflict management is the process to restrict the negative features of conflict while increasing the positive features of conflict. The goal of conflict management is to improve learning and group results including efficacy or performance in an organizational setting. Properly managed conflict can enhance group results.

2. Risk Management

Risk management is the analysis and identification of risks that is followed by synchronized and economical implementation of resources to minimize, operate and control the possibility or effect of unfortunate events or to maximize the realization of opportunities.

3. Requirement Management

It is the process of analyzing, prioritizing, tracking, and documenting requirements and then supervising change and communicating to pertinent stakeholders. It is a continuous process during a project.

4. Change Management

Change management is a systematic approach to dealing with the transition or transformation of an organization's goals, processes, or technologies. The purpose of change management is to execute strategies for effecting change, controlling change, and helping people to adapt to change.

5. Software Configuration Management

Software configuration management is the process of controlling and tracking changes in the software, part of the larger cross-disciplinary field of configuration management. Software configuration management includes revision control and the inauguration of baselines.

6. Release Management

Release Management is the task of planning, controlling, and scheduling the built-in deploying releases. Release management ensures that the organization delivers new and enhanced services required by the customer while protecting the integrity of existing services.

COCOMO Model: -

The Constructive Cost Model (COCOMO) is a software cost estimation model that helps predict the effort, cost, and schedule required for a software development project. Developed by Barry Boehm in 1981, COCOMO uses a mathematical formula based on the size of the software project, typically measured in lines of code (LOC).

The COCOMO Model is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality. It was proposed by Barry Boehm in 1981 and is based on the study of 63 projects, which makes it one of the best-documented models.

The key parameters that define the quality of any software product, which are also an outcome of COCOMO, are primarily effort and schedule:

1. **Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.

2. **Schedule:** This simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, and months.

Types of Projects in the COCOMO Model

In the COCOMO model, software projects are categorized into three types based on their complexity, size, and the development environment. These types are:

1. **Organic:** A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

2. **Semi-detached**: A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environments lie in between organic and embedded. The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience better guidance and creativity. Eg: Compilers or different Embedded Systems can be considered Semi-Detached types.

3. **Embedded:** A software project requiring the highest level of complexity, creativity, and experience requirement falls under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

Comparison of these three types of Projects in COCOMO Model

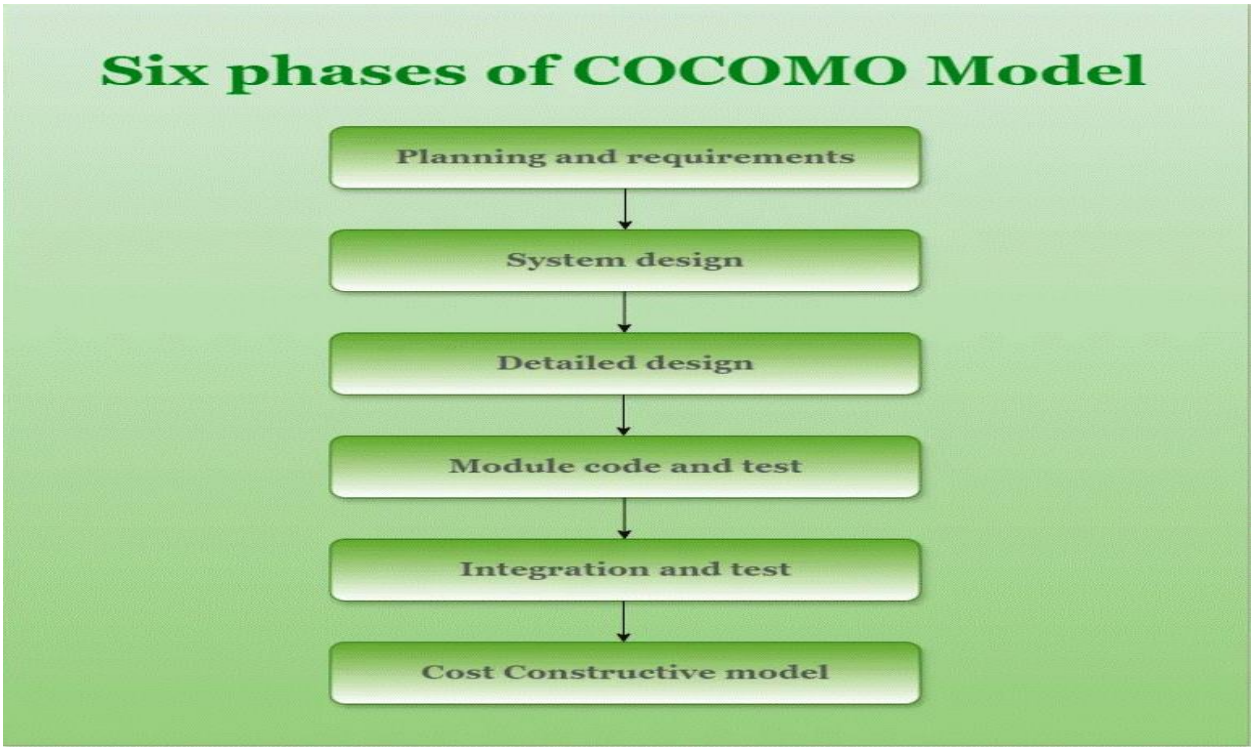
Aspects	Organic	Semidetached	Embedded
Project Size	2 to 50 KLOC	50-300 KLOC	300 and above KLOC
Complexity	Low	Medium	High
Team Experience	Highly experienced	Some experienced as well as inexperienced staff	Mixed experience, includes experts
Environment	Flexible, fewer constraints	Somewhat flexible, moderate constraints	Highly rigorous, strict requirements
Effort Equation	$E = 2.4(400)^{1.05}$	$E = 3.0(400)^{1.12}$	$E = 3.6(400)^{1.20}$
Example	Simple payroll system	New system interfacing with existing systems	Flight control software

Detailed Structure of COCOMO Model

Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the software engineering process. The detailed model uses different effort multipliers for each cost driver attribute. In detailed COCOMO, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System design
3. Detailed design
4. Module code and test
5. Integration and test
6. Cost Constructive model



Importance of the COCOMO Model

1. **Cost Estimation:** To help with resource planning and project budgeting, COCOMO offers a methodical approach to software development cost estimation.

2. **Resource Management:** By taking team experience, project size, and complexity into account, the model helps with efficient resource allocation.

3. **Project Planning**: COCOMO assists in developing practical project plans that include attainable objectives, due dates, and benchmarks.

4. **Risk management**: Early in the development process, COCOMO assists in identifying and mitigating potential hazards by including risk elements.

5. **Support for Decisions**: During project planning, the model provides a quantitative foundation for choices about scope, priorities, and resource allocation.

6. **Benchmarking**: To compare and assess various software development projects to industry standards, COCOMO offers a benchmark.

7. **Resource Optimization:** The model helps to maximize the use of resources, which raises productivity and lowers costs.

Types of COCOMO Model

There are three types of COCOMO Model:

- **Basic COCOMO Model**
- **Intermediate COCOMO Model**
- **Detailed COCOMO Model**

1. Basic COCOMO Model

The Basic COCOMO model is a straightforward way to estimate the effort needed for a software development project. It uses a simple mathematical formula to

predict how many person-months of work are required based on the size of the project, measured in thousands of lines of code (KLOC).

It estimates effort and time required for development using the following expression:

$E = a*(KLOC)^b$ PM

$Tdev = c*(E)^d$

Person required = Effort/ Time

Where,

E is effort applied in Person-Months

KLOC is the estimated size of the software product indicate in Kilo Lines of Code

Tdev is the development time in months

a, b, c are constants determined by the category of software project given in below table.

. Intermediate COCOMO Model

The basic COCOMO model assumes that the effort is only a function of the number of lines of code and some constants evaluated according to the different software systems. However, in reality, no system's effort and schedule can be solely calculated based on Lines of Code. For that, various other factors such as reliability, experience, and Capability. These factors are known as **Cost Drivers (multipliers)** and the Intermediate Model utilizes 15 such drivers for cost estimation.

Classification of Cost Drivers and their Attributes:

The cost drivers are divided into four categories

Product attributes:

- Required software reliability extent

- Size of the application database
- The complexity of the product

Hardware attributes

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

Personal attributes

- Analyst capability
- Software engineering capability
- Application experience
- Virtual machine experience
- Programming language experience

Project attributes

- Use of software tools
- Application of software engineering methods
- Required development schedule

Intermediate COCOMO Model equation:

$E = a*(KLOC)^b * EAF\ PM$

$Tdev = c*(E)^d$

Where,

- E is effort applied in Person-Months
- KLOC is the estimated size of the software product indicate in Kilo Lines of Code
- EAF is the Effort Adjustment Factor (EAF) is a multiplier used to refine the effort estimate obtained from the basic COCOMO model.
- Tdev is the development time in months
- a, b, c are constants determined by the category of software project given in below table.

3. Detailed COCOMO Model

Detailed COCOMO goes beyond Basic and Intermediate COCOMO by diving deeper into project-specific factors. It considers a wider range of parameters, like team experience, development practices, and software complexity. By analyzing these factors in more detail, Detailed COCOMO provides a highly accurate estimation of effort, time, and cost for software projects.

Advantages of the COCOMO Model

1. **Systematic cost estimation:** Provides a systematic way to estimate the cost and effort of a software project.
2. **Helps to estimate cost and effort:** This can be used to estimate the cost and effort of a software project at different stages of the development process.
3. **Helps in high-impact factors:** Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
4. **Helps to evaluate the feasibility of a project:** This can be used to evaluate the feasibility of a software project by estimating the cost and effort required to complete it.

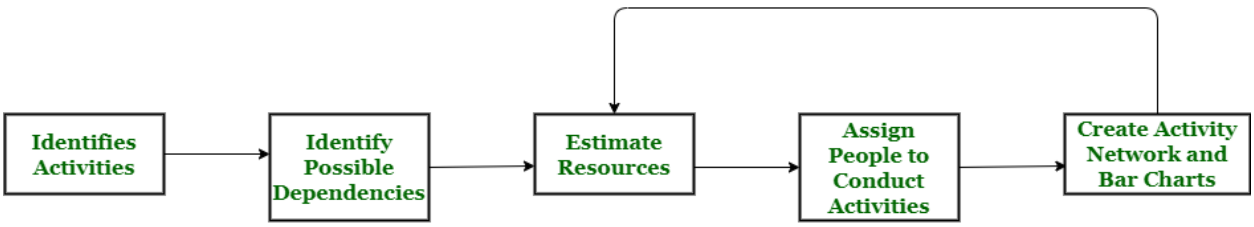
Disadvantages of the COCOMO Model

1. **Assumes project size as the main factor:** Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.
2. **Does not count development team-specific characteristics:** Does not take into account the specific characteristics of the development team, which can have a significant impact on the cost and effort of a software project.
3. **Not enough precise cost and effort estimate:** This does not provide a precise estimate of the cost and effort of a software project, as it is based on assumptions and averages.

Project Scheduling: -

Project schedule simply means a mechanism that is used to communicate and know about that tasks are needed and has to be done or performed and which organizational resources will be given or allocated to these tasks and in what time duration or time frame work is needed to be performed. Effective project scheduling leads to success of project, reduced cost, and increased customer satisfaction.

Scheduling in project management means to list out activities, deliverables, and milestones within a project that are delivered. It contains more notes than your average weekly planner notes. The most common and important form of project schedule is Gantt chart.



Project Scheduling Process

Process :

The manager needs to estimate time and resources of project while scheduling project. All activities in project must be arranged in a coherent sequence that means activities should be arranged in a logical and well-organized manner for easy to understand.

The total work is separated or divided into various small activities or tasks during project schedule. Then, Project manager will decide time required for each activity or task to get completed. Even some activities are conducted and performed in parallel for efficient performance. The project manager should be aware of fact that each stage of project is not problem-free.

Problems arise during Project Development Stage :

- People may leave or remain absent during particular stage of development.
- Hardware may get failed while performing.
- Software resource that is required may not be available at present, etc.

Resources required for Development of Project :

- Human effort
- Sufficient disk space on server
- Specialized hardware
- Software technology
- Travel allowance required by project staff, etc.

Advantages of Project Scheduling :

There are several advantages provided by project schedule in our project management:

- It simply ensures that everyone remains on same page as far as tasks get completed, dependencies, and deadlines.
- It helps in identifying issues early and concerns such as lack or unavailability of resources.
- It also helps to identify relationships and to monitor process.
- It provides effective budget management and risk mitigation.

Software Configuration Management: -

- The result of a large software development effort typically consist
  - of a large number of objects **e.g. source e code, design documents, SRS documents, test cases** etc.
- These objects are usually referred to and modify by a number of software engineers during the development phase of the software.
- The state of any of these objects at any point of time is called configuration.
- The state of an object changes as development progresses and also as bugs are detected and fixed.

  - **Requirement of Software**
  - **Configuration Management**

There are several reasons for putting an object under configuration management.

1. Inconsistency problem when the object is replicated.
2. Problems associated with concurrent access
3. Providing a stable development environment
4. System accounting and maintaining status

Existence of variant of a software product causes at least two kinds of problems.

(i) Suppose you have several alternate of the same module and final a bug in one of them. Then it has to be fixed in all the versions.

(ii) Suppose two different programmers are working on the same module to add different functionalities, them the changes carried out by different with each other.

- A configuration management tool handles these problems therefore configuration management tool helps keep track of the various objects, so that we can quickly and unambiguously determine the current state of the product.

- "The set of activities by which a large number of objects are managed and maintained is termed as configuration management."

Configuration Management Activities

Configuration management is implemented through two principal activities:

- Configuration identification decides the part of the system, which needs to be kept track of
- Configuration control ensures that changes to a system are implemented smoothly.

Configuration Identification

Any object associated with a software development activity can be classification into three main categories:

- Controlled
- Pre-controlled
- Uncontrolled

Controlled

- Objects are under configuration control and some formal procedures must be followed to change them.

Pre-controlled

- Objects are not yet under configuration control, but will be eventually under configuration control.

Uncontrolled

Objects are not and will not be subject to configuration control.

Typical Controllable objects include:

- Requirement specification document
- Design documents
- Tools used to build the system, such as compilers, linkers, lexical analyzers, parsers etc.

- Source code for each module
- Test case
- Problem reports

- Configuration management plan written during the project planning phase lists all controlled objects.
- The managers who develop the plan must strike a balance between controlling too much and controlling too little.

Configuration Control

- Configuration Control is the process of managing changes to controlled objects.
- In order to change a module, a developed can get a private copy of the module by the reverse operation.
- Configuration management tools allow any one person to reserve a module and do not allow anyone else to reserve this module until the reserved module is restored.
- Thus, by not allowing more than one engineer to simultaneously reserve a module, the problems associated with concurrent access are solved.

- **Change is well motivated.**
- **The developer has considered and documented the effects of the change.**
- **Changes interact well with the changes mode by other developers.**
- **Appropriate people have a validated the change.**

Note: At any time, a programmer may pay attention to two versions of a configuration.

(i) Current base Line Configuration
(ii) Programmer's Private Version

SOFTWARE MAINTENANCE:-

- Once the software is delivered and deployed, it enters the maintenance phase.
- All systems need maintenance, but for others systems it is largely due to problems that are introduced to aging.
- Why is maintenance needed for software, when software does not age?
- Software needs to be maintained not because some residual errors remaining in the system that must be removed as they are discovered.
- Software maintenance is becoming an important activity of a large number of organizations.
- This is no surprise, given the rate of hardware obsolescence, the immortality of a software product per se, and the demand of the user community to see the existing software products run on newer platforms, run in newer environment, and/or with enhanced features.
- Whenever change in hardware, change in support environment, maintenance is needed. For instance, a software product may need to be maintained when the operating system changes.

Types of Software maintenance

There are mainly three category of maintenance

Corrective

- Corrective maintenance of a software product may be necessary either to correct some bugs observed while the system is in use or to improve the performance of the system.

Adaptive

- A software product might need adaptive maintenance when the customers need the product to run on new platforms or new operating system or when they need the product to interface with new hardware/ software.

Perfective

- To support the new features that the users want or to change different functionalities of the system according to user.
- Software maintenance has become an important activity of a large number of organizations.
- When the hardware platform changes and a software product perform some low level functions, maintenance is necessary.
- Also, whenever the software supports environment changes, the software product requires rework to cope with the new interface. Thus every software product continues to evolve after its development through maintenance efforts.

The activities involved in a software maintenance projects are not unique and depend on several factors such as

(i) The extent of modification to the product required
(ii) The resource available to the maintenance team
(iii) The conditions of the existing product
(iv) The expected project risks

For complex maintenance projects, the software process can be represented by a forward engineering cycle with emphasis on as much reuse as possible from the existing code and other documents.

QUALITY ASSURANCE PLANS:-

- To ensure that final product produced is of high quality, some quality control activities must be performed throughout the development.
- The purpose of the software quality assurance plan(SQAP) is to specify, all the work product that need to be produced during the project, activities that need to be performed for checking the quality of each of the work products and the tool and method that may used for SQA activities.
- To ensure that the delivered software is of good quality, it is essential to make sure that requirement and design are also of good quality.
- The documents that should be produced during software development to enhance software quality should be specified by the SQAP.
- It should identify all documents that govern the development, verification, validation use and maintenance of the software and how these documents are to be checked for adequacy.

Verification and Validation:-

Verification and Validation is the process of investigating whether a software system satisfies specifications and standards and fulfills the required purpose. **Barry Boehm** described verification and validation as the following:

Verification: Are we building the product right?

Validation: Are we building the right product?

Verification

Verification is the process of checking that software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfills the requirements that we have. Verification is simply known as **Static Testing.**

Static Testing

Verification Testing is known as Static Testing and it can be simply termed as checking whether we are developing the right product or not and also whether our software is fulfilling the customer's requirement or not. Here are some of the activities that are involved in verification.

- Inspections
- Reviews
- Walkthroughs
- Desk-checking

Validation

Validation is the process of checking whether the software product is up to the mark or in other words product has high-level requirements. It is the process of checking the validation of the product i.e. it checks what we are developing is the right product. it is a validation of actual and expected products. Validation is simply known as **Dynamic Testing.**

Dynamic Testing

Validation Testing is known as Dynamic Testing in which we examine whether we have developed the product right or not and also about the business needs of the client. Here are some of the activities that are involved in Validation.

1. Black Box Testing
2. White Box Testing
3. Unit Testing
4. Integration Testing

Project Monitoring Plans: -

Monitoring in project management is the systematic process of observing, measuring, and evaluating activities, resources, and progress to verify that a given asset has been developed according to the terms set out. It is intended to deliver instant insights, detect deviations from the plan, and allow quick decision-making.

Purpose

1. **Track Progress:** Monitor the actual implementation of the project along with indicators such as designs, timelines budgets, and standards.

2. **Identify Risks and Issues:** Identify other risks and possible issues in the early stage to create immediate intervention measures as well as resolutions.

3. **Ensure Resource Efficiency:** Monitor how resources are being distributed and used to improve efficiency while avoiding resource shortages.

4. **Facilitate Decision-Making:** Supply project managers and stakeholders with reliable and timely information for informed

5. **Enhance Communication:** Encourage honest team communication and stakeholder engagement related to project status, challenges

Key Activities

1. **Performance Measurement:** Identify and monitor critical performance indicators (KPIs) to compare the progress of a project against defined targets.

2. **Progress Tracking:** Update schedules and timelines for the project on a regular basis, and compare actual work with planned milestones to detect any delays or deviations.

3. **Risk Identification and Assessment:** Monitor actual risks, including their probability and consequences. Find new risks and assess the performance of current risk mitigation mechanisms.

4. **Issue Identification and Resolution:** Point out problems discovered in the process of project implementation, evaluate their scale and introduce corrective measures immediately.

5. **Resource Monitoring:** Track how resources are distributed and used, to ensure there is adequate equipment as well as support by the team members in meeting their objectives.

6. **Quality Assurance:** Monitor compliance with quality standards and processes, reporting deviations to take actions necessary for restoring the targeted level of quality.

7. **Communication and Reporting:** Disseminate project status updates, milestones reached and important findings to the stakeholders on a regular basis.

8. **Change Control:** Review and evaluate project scope, schedule or budget changes. Adopt structured change control processes to define, justify and approve changes.

9. **Documentation Management:** Make sure that project documentation is accurate, current and readily available for ready reference. This involves project plans, reports and other documents related to a particular project.

Tools and Technologies for Monitoring

1. **Project Management Software:** Tools such as Microsoft Project, Jira, and Trello offer features in terms of scheduling monitoring resources for task execution.

2. **Performance Monitoring Tools:** The solutions that New Relic, AppDynamics and Dynatrace provide cater to monitoring of application performances as well as infrastructure performance besides user experience.

3. **Network Monitoring Tools:** The three tools namely SolarWinds Network Performance Monitor, Wireshark and PRTG Network monitor help in monitoring and analyzing the network performance.

4. **Server and Infrastructure Monitoring Tools:** The mentioned monitoring tools, namely Nagios prometheus and Zabbix monitor servers systems and IT infrastructure for performance availability.

5. **Log Management Tools:** Log analysis and visualization are performed using ELK Stack (Elasticsearch, Logstash, Kibana), Splunk, and Graylog.

6. **Cloud Monitoring Tools:** Amazon CloudWatch, Google Cloud Operations Suite, and Azure Monitor provide monitoring solutions for cloud-based services and resources.

7. **Security Monitoring Tools:** Security Information and Event Management tools like Splunk, IBM QRadar or ArcSight provide support to the process of monitoring security events and incidents.

Software Risk Management: -

What is Risk Management?

Risk Management is a systematic process of recognizing, evaluating, and handling threats or risks that have an effect on the finances, capital, and overall operations of an organization. These risks can come from different areas, such as financial instability, legal issues, errors in strategic planning, accidents, and natural disasters.

Why is risk management important?

Risk management is important because it helps organizations to prepare for unexpected circumstances that can vary from small issues to major crises. By actively understanding, evaluating, and planning for potential risks, organizations can protect their financial health, continued operation, and overall survival.

Suppose In a software development project, one of the key developers unexpectedly falls ill and is unable to contribute to the product for an extended period.

One of the solution that organization may have , The team uses collaborative tools and procedures, such as shared work boards or project management software, to make sure that each member of the team is aware of all tasks and responsibilities, including those of their teammates.

The risk management process

Risk management is a sequence of steps that help a software team to understand, analyze, and manage uncertainty. Risk management process consists of

- Risks Identification.
- Risk Assessment.
- Risks Planning.
- Risk Monitoring

Risk Identification

Risk identification refers to the systematic process of recognizing and evaluating potential threats or hazards that could negatively impact an organization, its operations, or its workforce. This involves identifying various types of risks, ranging from IT security threats like viruses and phishing attacks to unforeseen events such as equipment failures and extreme weather conditions.

Risk analysis

Risk analysis is the process of evaluating and understanding the potential impact and likelihood of identified risks on an organization. It helps determine how serious a risk is and how to best manage or mitigate it. Risk Analysis involves evaluating each risk's probability and potential consequences to prioritize and manage them effectively.

Risk Planning

Risk planning involves developing strategies and actions to manage and mitigate identified risks effectively. It outlines how to respond to potential risks, including prevention, mitigation, and contingency measures, to protect the organization's objectives and assets.

Risk Monitoring

Risk monitoring involves continuously tracking and overseeing identified risks to assess their status, changes, and effectiveness of mitigation strategies. It ensures that risks are regularly reviewed and managed to maintain alignment with organizational objectives and adapt to new developments or challenges.

Understanding Risks in Software Projects

A computer code project may be laid low with an outsized sort of risk. To be ready to consistently establish the necessary risks that could affect a computer code project, it's necessary to group risks into completely different categories.

There are mainly 3 classes of risks that may affect a computer code project:

1. **Project Risks:**
   Project risks concern various sorts of monetary funds, schedules, personnel, resources, and customer-related issues. A vital project risk is schedule slippage. Since computer code is intangible, it's tough to observe and manage a computer code project. It's tough to manage one thing that can not be seen. For any producing project, like producing cars, the project manager will see the merchandise taking form.

   For example, see that the engine is fitted, at the moment the area of the door unit is fitted, the automotive is being painted, etc. so he will simply assess the progress of the work and manage it. The physical property of the

merchandise being developed is a vital reason why several computer codes come to suffer from the danger of schedule slippage.

2. **Technical Risks:**
   Technical risks concern potential style, implementation, interfacing, testing, and maintenance issues. Technical risks conjointly embody ambiguous specifications, incomplete specifications, dynamic specifications, technical uncertainty, and technical degeneration. Most technical risks occur thanks to the event team's lean information concerning the project.

3. **Business Risks:**
   This type of risk embodies the risks of building a superb product that nobody needs, losing monetary funds or personal commitments, etc.

Classification of Risk in a project

Example: Let us consider a satellite-based mobile communication project. The project manager can identify many risks in this project. Let us classify them appropriately.

- What if the project cost escalates and overshoots what was estimated? – **Project Risk**
- What if the mobile phones that are developed become too bulky to conveniently carry? **Business Risk**
- What if call hand-off between satellites becomes too difficult to implement? **Technical Risk**

Project Planning: -

A Software Project is the complete methodology of programming advancement from requirement gathering to testing and support, completed by the execution procedures, in a specified period to achieve intended software product.

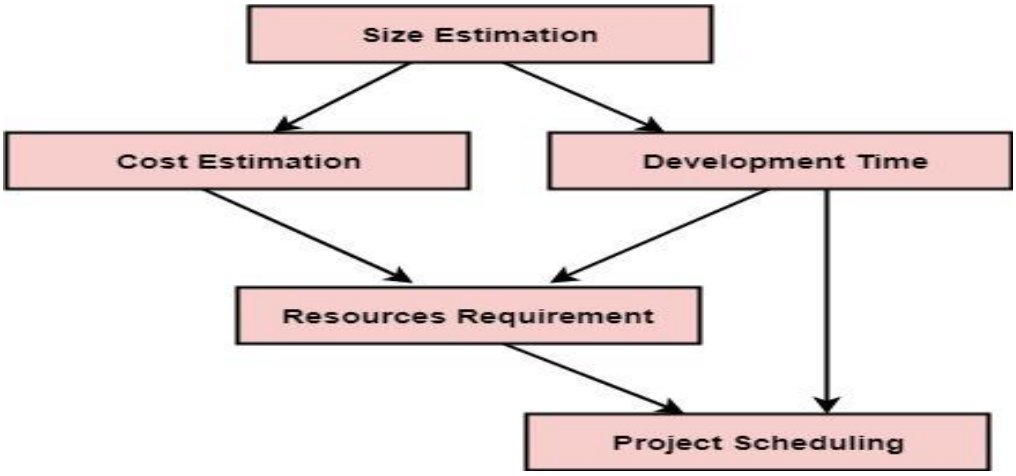
Need of Software Project Management

Software development is a sort of all new streams in world business, and there's next to no involvement in structure programming items. Most programming items are customized to accommodate customer's necessities. The most significant is that the underlying technology changes and advances so generally and rapidly that experience of one element may not be connected to the other one. All such business and ecological imperatives bring risk in software development; hence, it is fundamental to manage software projects efficiently.

Software Project Manager

Software manager is responsible for planning and scheduling project development. They manage the work to ensure that it is completed to the required standard. They monitor the progress to check that the event is on time and within budget. The project planning must incorporate the major issues like size & cost estimation scheduling, project monitoring, personnel selection evaluation & risk management. To plan a successful software project, we must understand:

- Scope of work to be completed
- Risk analysis
- The resources mandatory
- The project to be accomplished
- Record of being followed



Software Reliability and Quality Assurance:-

Software Reliability

Software Reliability means **Operational reliability**. It is described as the ability of a system or component to perform its required functions under static conditions for a specific period.

Software reliability is also defined as the probability that a software system fulfills its assigned task in a given environment for a predefined number of input cases, assuming that the hardware and the input are free of error.

Software Reliability is an essential connect of software quality, composed with functionality, usability, performance, serviceability, capability, installability, maintainability, and documentation. Software Reliability is hard to achieve because the complexity of software turn to be high. While any system with a high degree of complexity, containing software, will be hard to reach a certain level of reliability, system developers tend to push complexity into the software layer, with the speedy growth of system size and ease of doing so by upgrading the software.

Software Quality Assurance

What is Quality?

Quality defines to any measurable characteristics such as correctness, maintainability, portability, testability, usability, reliability, efficiency, integrity, reusability, and interoperability.

There are two kinds of Quality:

Quality of Design: Quality of Design refers to the characteristics that designers specify for an item. The grade of materials, tolerances, and performance specifications that all contribute to the quality of design.

Quality of conformance: Quality of conformance is the degree to which the design specifications are followed during manufacturing. Greater the degree of conformance, the higher is the level of quality of conformance.

Software Quality: Software Quality is defined as the conformance to explicitly state functional and performance requirements, explicitly documented development standards, and inherent characteristics that are expected of all professionally developed software.

Quality Control: Quality Control involves a series of inspections, reviews, and tests used throughout the software process to ensure each work product meets the requirements place upon it. Quality control includes a feedback loop to the process that created the work product.

Quality Assurance: Quality Assurance is the preventive set of activities that provide greater confidence that the project will be completed successfully.

Quality Assurance focuses on how the engineering and management activity will be done?

As anyone is interested in the quality of the final product, it should be assured that we are building the right product.

It can be assured only when we do inspection & review of intermediate products, if there are any bugs, then it is debugged. This quality can be enhanced.

Importance of Quality

We would expect the quality to be a concern of all producers of goods and services. However, the distinctive characteristics of software and in particular its intangibility and complexity, make special demands.

Increasing criticality of software: The final customer or user is naturally concerned about the general quality of software, especially its reliability. This is increasing in the case as organizations become more dependent on their

computer systems and software is used more and more in safety-critical areas. For example, to control aircraft.

The intangibility of software: This makes it challenging to know that a particular task in a project has been completed satisfactorily. The results of these tasks can be made tangible by demanding that the developers produce 'deliverables' that can be examined for quality.

Accumulating errors during software development: As computer system development is made up of several steps where the output from one level is input to the next, the errors in the earlier ?deliverables? will be added to those in the later stages leading to accumulated determinable effects. In general the later in a project that an error is found, the more expensive it will be to fix. In addition, because the number of errors in the system is unknown, the debugging phases of a project are particularly challenging to control.

Software Quality Assurance

Software quality assurance is a planned and systematic plan of all actions necessary to provide adequate confidence that an item or product conforms to establish technical requirements.

A set of activities designed to calculate the process by which the products are developed or manufactured.

- A quality management approach
- Effective Software engineering technology (methods and tools)
- Formal technical reviews that are tested throughout the software process
- A multitier testing strategy
- Control of software documentation and the changes made to it.
- A procedure to ensure compliances with software development standards
- Measuring and reporting mechanisms.

Quality Assurance v/s Quality control

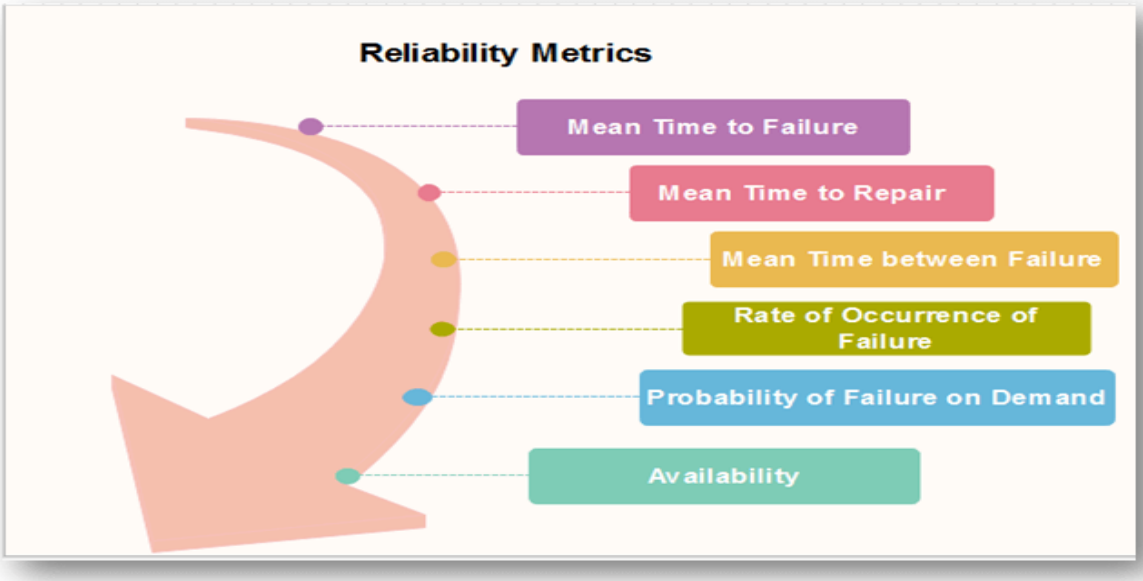
Quality Assurance	Quality Control
Quality Assurance (QA) is the set of actions including facilitation, training,	**Quality Control (QC)** is described as the processes and methods used to

measurement, and analysis needed to provide adequate confidence that processes are established and continuously improved to produce products or services that conform to specifications and are fit for use.	compare product quality to requirements and applicable standards, and the actions are taken when a nonconformance is detected.
QA is an activity that establishes and calculates the processes that produce the product. If there is no process, there is no role for QA.	**QC** is an activity that demonstrates whether or not the product produced met standards.
QA helps establish process	**QC** relates to a particular product or service
QA sets up a measurement program to evaluate processes	**QC** verified whether particular attributes exist, or do not exist, in a explicit product or service.
QA identifies weakness in processes and improves them	**QC** identifies defects for the primary goals of correcting errors.
Quality Assurance is a managerial tool.	Quality Control is a corrective tool.
Verification is an example of QA.	Validation is an example of QC.

Reliability metrics: -

Reliability metrics are used to quantitatively expressed the reliability of the software product. The option of which metric is to be used depends upon the type of system to which it applies & the requirements of the application domain.

Some reliability metrics which can be used to quantify the reliability of the software product are as follows:



1. Mean Time to Failure (MTTF)

MTTF is described as the time interval between the two successive failures. An **MTTF** of 200 mean that one failure can be expected each 200-time units. The time units are entirely dependent on the system & it can even be stated in the number of transactions. **MTTF** is consistent for systems with large transactions.

For example, It is suitable for computer-aided design systems where a designer will work on a design for several hours as well as for Word-processor systems.

To measure **MTTF**, we can evidence the failure data for n failures. Let the failures appear at the time instants $\mathbf{t_1, t_2 ..... t_n}$.

MTTF can be calculated as

$$\sum_{i=1}^{n} \frac{t_{i+1} - t_i}{(n-1)}$$

2. Mean Time to Repair (MTTR)

Once failure occurs, some-time is required to fix the error. **MTTR** measures the average time it takes to track the errors causing the failure and to fix them.

3. Mean Time Between Failure (MTBR)

We can merge **MTTF** & **MTTR** metrics to get the MTBF metric.

MTBF = MTTF + MTTR

Thus, an **MTBF** of 300 denoted that once the failure appears, the next failure is expected to appear only after 300 hours. In this method, the time measurements are real-time & not the execution time as in **MTTF**.

4. Rate of occurrence of failure (ROCOF)

It is the number of failures appearing in a unit time interval. The number of unexpected events over a specific time of operation. **ROCOF** is the frequency of occurrence with which unexpected role is likely to appear. A **ROCOF** of 0.02 mean that two failures are likely to occur in each 100 operational time unit steps. It is also called the failure intensity metric.

5. Probability of Failure on Demand (POFOD)

POFOD is described as the probability that the system will fail when a service is requested. It is the number of system deficiency given several systems inputs.

POFOD is the possibility that the system will fail when a service request is made.

A **POFOD** of 0.1 means that one out of ten service requests may fail.**POFOD** is an essential measure for safety-critical systems. POFOD is relevant for protection systems where services are demanded occasionally.

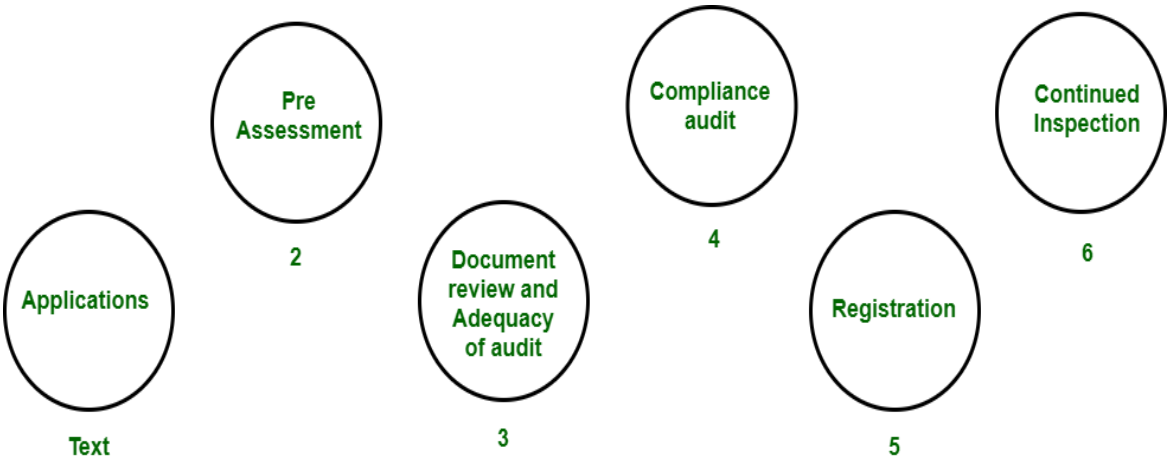
6. Availability (AVAIL)

Availability is the probability that the system is applicable for use at a given time. It takes into account the repair time & the restart time for the system. An availability of 0.995 means that in every 1000 time units, the system is feasible to be available for **995** of these. The percentage of time that a system is applicable for use, taking into account planned and unplanned downtime. If a system is down an average of four hours out of 100 hours of operation, its **AVAIL** is 96%.

ISO-9000 Certification for Software industry:-

The International organization for Standardization is a world wide federation of national standard bodies. The **International standards organization (ISO)** is a standard which serves as a for contract between independent parties. It specifies guidelines for development of **quality system**.

Quality system of an organization means the various activities related to its products or services. Standard of ISO addresses to both aspects i.e. operational and organizational aspects which includes responsibilities, reporting etc. An ISO 9000 standard contains set of guidelines of production process without considering product itself.



ISO 9000 Certification

Why ISO Certification required by Software Industry?
There are several reasons why software industry must get an ISO certification. Some of reasons are as follows :

- This certification has become a standards for international bidding.
- It helps in designing high-quality repeatable software products.
- It emphasis need for proper documentation.
- It facilitates development of optimal processes and totally quality measurements.

Features of ISO 9001 Requirements :

- **Document control –**
  All documents concerned with the development of a software product should be properly managed and controlled.
- **Planning –**
  Proper plans should be prepared and monitored.
- **Review –**
  For effectiveness and correctness all important documents across all phases should be independently checked and reviewed .
- **Testing –**
  The product should be tested against specification.
- **Organizational Aspects –**
  Various organizational aspects should be addressed e.g., management reporting of the quality team.

Advantages of ISO 9000 Certification :
Some of the advantages of the ISO 9000 certification process are following :

- Business ISO-9000 certification forces a corporation to specialize in “how they are doing business”. Each procedure and work instruction must be documented and thus becomes a springboard for continuous improvement.
- Employees morale is increased as they’re asked to require control of their processes and document their work processes
- Better products and services result from continuous improvement process.
- Increased employee participation, involvement, awareness and systematic employee training are reduced problems.

Shortcomings of ISO 9000 Certification :
Some of the shortcoming of the ISO 9000 certification process are following :

- ISO 9000 does not give any guideline for defining an appropriate process and does not give guarantee for high quality process.
- ISO 9000 certification process have no international accreditation agency exists.

Software measurement and metrics: -

Software Measurement: A measurement is a manifestation of the size, quantity, amount, or dimension of a particular attribute of a product or process. Software measurement is a titrate impute of a characteristic of a software product or the software process.

Software Measurement Principles

The software measurement process can be characterized by five activities-

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
3. **Analysis:** The computation of metrics and the application of mathematical tools.
4. **Interpretation:** The evaluation of metrics results in insight into the quality of the representation.
5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

Need for Software Measurement

Software is measured to:

- Create the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Enhance the quality of a product or process.
- Regulate the state of the project concerning budget and schedule.
- Enable data-driven decision-making in project planning and control.

- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.
- Give software products and processes a quantitative basis for evaluation.
- Enable the ongoing improvement of software development practices.

Classification of Software Measurement

There are 2 types of software measurement:

1. **Direct Measurement:** In direct measurement, the product, process, or thing is measured directly using a standard scale.
2. **Indirect Measurement:** In indirect measurement, the quantity or quality to be measured is measured using related parameters i.e. by use of reference.

Software Metrics

A metric is a measurement of the level at which any impute belongs to a system product or process.

Software metrics are a quantifiable or countable assessment of the attributes of a software product. There are 4 functions related to software metrics:

1. **Planning**
2. **Organizing**
3. **Controlling**
4. **Improving**

Characteristics of software Metrics

1. **Quantitative:** Metrics must possess a quantitative nature. It means metrics can be expressed in numerical values.

2. **Understandable:** Metric computation should be easily understood, and the method of computing metrics should be clearly defined.

3. **Applicability:** Metrics should be applicable in the initial phases of the development of the software.

4. **Repeatable:** When measured repeatedly, the metric values should be the same and consistent.

5. **Economical:** The computation of metrics should be economical.

6. **Language Independent:** Metrics should not depend on any programming language.

Types of Software Metrics

1. **Product Metrics:** Product metrics are used to evaluate the state of the product, tracing risks and undercover prospective problem areas. The ability of the team to control quality is evaluated. Examples include lines of code, cyclomatic complexity, code coverage, defect density, and code maintainability index.

2. **Process Metrics:** Process metrics pay particular attention to enhancing the long-term process of the team or organization. These metrics are used to optimize the development process and maintenance activities of software. Examples include effort variance, schedule variance, defect injection rate, and lead time.

3. **Project Metrics:** The project metrics describes the characteristic and execution of a project. Examples include effort estimation accuracy, schedule deviation, cost variance, and productivity. Usually measures-

    - Number of software developer
    - Staffing patterns over the life cycle of software
    - Cost and schedule
    - Productivity

Advantages of Software Metrics

1. Reduction in cost or budget.
2. It helps to identify the particular area for improvising.
3. It helps to increase the product quality.
4. Managing the workloads and teams.
5. Reduction in overall time to produce the product,.
6. It helps to determine the complexity of the code and to test the code with resources.
7. It helps in providing effective planning, controlling and managing of the entire product.

Disadvantages of Software Metrics

1. It is expensive and difficult to implement the metrics in some cases.
2. Performance of the entire team or an individual from the team can't be determined. Only the performance of the product is determined.
3. Sometimes the quality of the product is not met with the expectation.
4. It leads to measure the unwanted data which is wastage of time.
5. Measuring the incorrect data leads to make wrong decision making.

SEI Capability Maturing Model:-

The Capability Maturity Model (CMM) is a procedure used to develop and refine an organization's software development process.

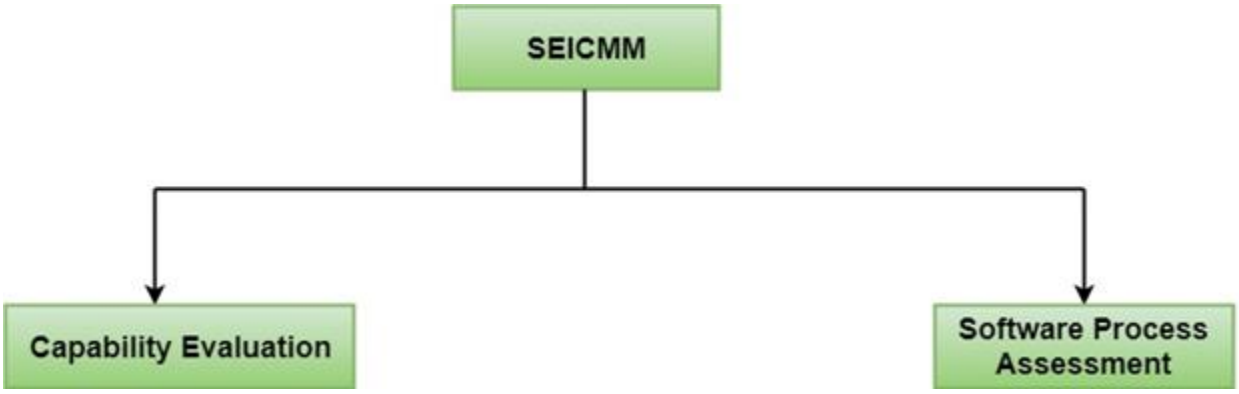
The model defines a five-level evolutionary stage of increasingly organized and consistently more mature processes.

CMM was developed and is promoted by the Software Engineering Institute (SEI), a research and development center promote by the U.S. Department of Defense (DOD).

Capability Maturity Model is used as a benchmark to measure the maturity of an organization's software process.

Methods of SEICMM

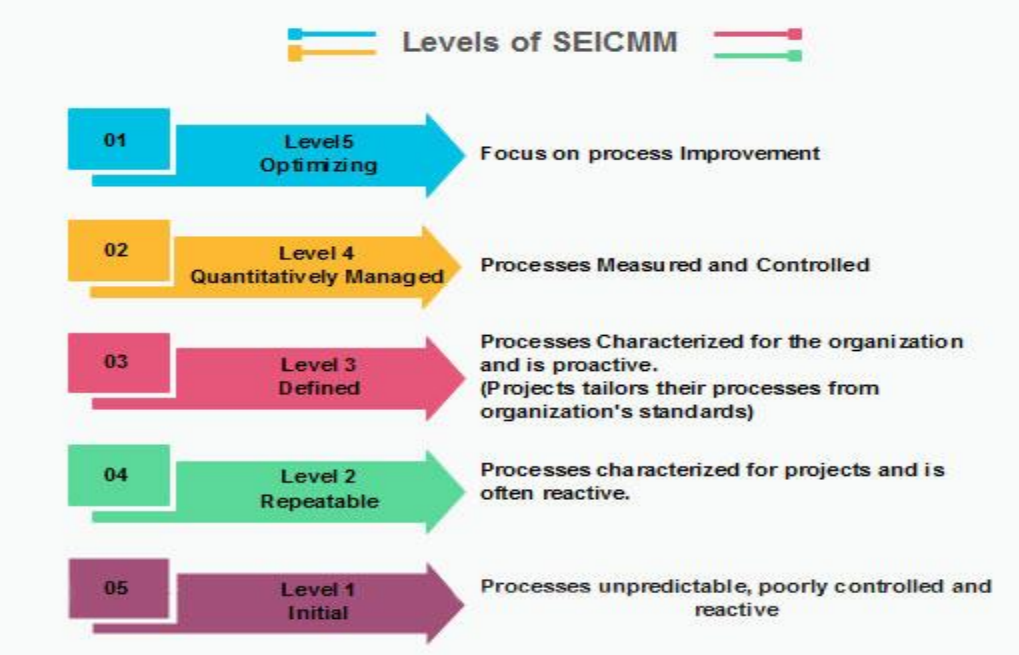
There are two methods of SEICMM:



Capability Evaluation: Capability evaluation provides a way to assess the software process capability of an organization. The results of capability evaluation indicate the likely contractor performance if the contractor is awarded a work. Therefore, the results of the software process capability assessment can be used to select a contractor.

Software Process Assessment: Software process assessment is used by an organization to improve its process capability. Thus, this type of evaluation is for purely internal use.

SEI CMM categorized software development industries into the following five maturity levels. The various levels of SEI CMM have been designed so that it is easy for an organization to build its quality system starting from scratch slowly.



Computer Aided Software Engineering SASE and its Scope:-

Computer-aided software engineering (CASE) is the implementation of computer-facilitated tools and methods in software development. CASE is used to ensure high-quality and defect-free software. CASE ensures a check-pointed and disciplined approach and helps designers, developers, testers, managers, and others to see the project milestones during development.

What is CASE Tools?

The essential idea of CASE tools is that in-built programs can help to analyze developing systems in order to enhance quality and provide better outcomes. Throughout the 1990, CASE tool became part of the software lexicon, and big companies like IBM were using these kinds of tools to help create software.

Types of CASE Tools:

1. **Diagramming Tools:** It helps in diagrammatic and graphical representations of the data and system processes. It represents system elements, control flow and data flow among different software components and system structures in a pictorial form. For example, Flow Chart Maker tool for making state-of-the-art flowcharts.

2. **Computer Display and Report Generators:** These help in understanding the data requirements and the relationships involved.

3. **Analysis Tools:** It focuses on inconsistent, incorrect specifications involved in the diagram and data flow. It helps in collecting requirements, automatically check for any irregularity, imprecision in the diagrams, data redundancies, or erroneous omissions.
   For example:

   - (i) Accept 360, Accompa, CaseComplete for requirement analysis.
   - (ii) Visible Analyst for total analysis.

4. **Central Repository:** It provides a single point of storage for data diagrams, reports, and documents related to project management.

5. **Documentation Generators:** It helps in generating user and technical documentation as per standards. It creates documents for technical users and end users.
   For example, Doxygen, DrExplain, Adobe RoboHelp for documentation.

6. **Code Generators:** It aids in the auto-generation of code, including definitions, with the help of designs, documents, and diagrams.

7. **Tools for Requirement Management:** It makes gathering, evaluating, and managing software needs easier.

8. **Tools for Analysis and Design**: It offers instruments for modelling system architecture and behaviour, which helps throughout the analysis and design stages of software development.

9. **Tools for Database Management:** It facilitates database construction, design, and administration.

10. **Tools for Documentation:** It makes the process of creating, organizing, and maintaining project documentation easier.

Advantages of the CASE approach:

- **Improved Documentation:** Comprehensive documentation creation and maintenance is made easier by CASE tools. Since automatically generated documentation is usually more accurate and up to date, there are fewer opportunities for errors and misunderstandings brought on by out-of-current material.

- **Reusing Components:** Reusable component creation and maintenance are frequently facilitated by CASE tools. This encourages a development approach that is modular and component-based, enabling teams to shorten development times and reuse tested solutions.

- **Quicker Cycles of Development:** Development cycles take less time when certain jobs, such testing and code generation, are automated. This may result in software solutions being delivered more quickly, meeting deadlines and keeping up with changing business requirements.

- **Improved Results**: Code generation, documentation, and testing are just a few of the time-consuming, repetitive operations that CASE tools perform. Due to this automation, engineers are able to concentrate on more intricate and imaginative facets of software development, which boosts output.

- **Achieving uniformity and standardization:** Coding conventions, documentation formats and design patterns are just a few of the areas of software development where CASE tools enforce uniformity and standards. This guarantees consistent and maintainable software development.

Disadvantages of the CASE approach:

- **Cost:** Using a case tool is very costly. Most firms engaged in software development on a small scale do not invest in CASE tools because they think that the benefit of CASE is justifiable only in the development of large systems.

- **Learning Curve:** In most cases, programmers' productivity may fall in the initial phase of implementation, because users need time to learn the technology. Many consultants offer training and on-site services that can be important to accelerate the learning curve and to the development and use of the CASE tools.

- **Tool Mix:** It is important to build an appropriate selection tool mix to urge cost advantage CASE integration and data integration across all platforms is extremely important.

CASE Tool and its Scope

A CASE (Computer power-assisted software package Engineering) tool could be a generic term accustomed to denote any type of machine-driven support for software package engineering. In a very additional restrictive sense, a CASE tool suggests that any tool accustomed to automatize some activity related to software package development.

Reasons for Using CASE Tools

The primary reasons for employing a CASE tool are:

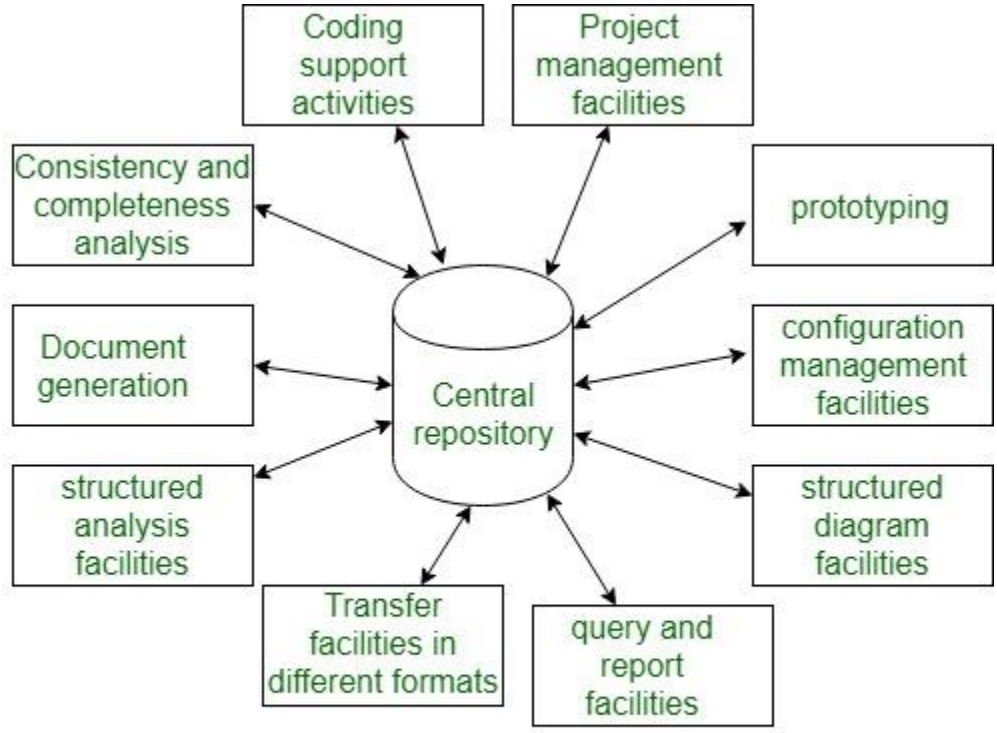
- To extend productivity
- To assist in turning out higher quality codes at a lower price

CASE Environment

Although individual CASE tools square measure helpful, the true power of a tool set is often completed only when this set of tools square measure integrated into a typical framework or setting.

1. CASE tools square measure characterized by the stage or stages of package development life cycle that they focus on.
2. Since different tools covering different stages share common data, it's needed that they integrate through some central repository to possess an even read of data related to the package development artifacts.
3. This central repository is sometimes information lexicon containing the definition of all composite and elementary data things.
4. Through the central repository, all the CASE tools in a very CASE setting share common data among themselves. therefore a CASE setting facilities the automation of the step-wise methodologies for package development.

A schematic illustration of a CASE setting is shown in the below diagram:



A CASE environment

Software Reverse Engineering:-

Software Reverse Engineering is a process of recovering the design, requirement specifications, and functions of a product from an analysis of its code. It builds a program database and generates information from this. This article focuses on discussing reverse engineering in detail.

What is Reverse Engineering?

Reverse engineering can extract design information from source code, but the abstraction level, the completeness of the documentation, the degree to which tools and a human analyst work together, and the directionality of the process are highly variable.

Objective of Reverse Engineering:

1. **Reducing Costs:** Reverse engineering can help cut costs in product development by finding replacements or cost-effective alternatives for systems or components.

2. **Analysis of Security:** Reverse engineering is used in cybersecurity to examine exploits, vulnerabilities, and malware. This helps in understanding of threat mechanisms and the development of practical defenses by security experts.

3. **Integration and Customization:** Through the process of reverse engineering, developers can incorporate or modify hardware or software components into pre-existing systems to improve their operation or tailor them to meet particular needs.

4. **Recovering Lost Source Code:** Reverse engineering can be used to recover the source code of a software application that has been lost or is inaccessible or at the very least, to produce a higher-level representation of it.

5. **Fixing bugs and maintenance:** Reverse engineering can help find and repair flaws or provide updates for systems for which the original source code is either unavailable or inadequately documented.

Reverse Engineering Goals:

1. **Cope with Complexity:** Reverse engineering is a common tool used to understand and control system complexity. It gives engineers the ability to analyze complex systems and reveal details about their architecture, relationships and design patterns.

2. **Recover lost information:** Reverse engineering seeks to retrieve as much information as possible in situations where source code or documentation are lost or unavailable. Rebuilding source code, analyzing data structures and retrieving design details are a few examples of this.

3. **Detect side effects:** Understanding a system or component's behavior requires analyzing its side effects. Unintended implications, dependencies, and interactions that might not be obvious from the system's

documentation or original source code can be found with the use of reverse engineering.

4. **Synthesis higher abstraction:** Abstracting low-level features in order to build higher-level representations is a common practice in reverse engineering. This abstraction makes communication and analysis easier by facilitating a greater understanding of the system's functionality.

5. **Facilitate Reuse:** Reverse engineering can be used to find reusable parts or modules in systems that already exist. By understanding the functionality and architecture of a system, developers can extract and repurpose components for use in other projects, improving efficiency and decreasing development time.

Reverse Engineering to Understand Data:

Reverse engineering of data occurs at different levels of abstraction .It is often the first reengineering task.

1. At the **program level**, internal program data structures must often be reverse engineered as part of an overall reengineering effort.

2. At the **system level**, global data structures (e.g., files, databases) are often reengineered to accommodate new database management paradigms (e.g., the move from flat file to relational or object-oriented database systems).

Internal Data Structures

Reverse engineering techniques for internal program data focus on the definition of classes of objects.

1. This is accomplished by examining the program code with the intent of grouping related program variables.

2. In many cases, the data organization within the code identifies abstract data types.

3. For example, record structures, files, lists, and other data structures often provide an initial indicator of classes.

Database Structures

A database allows the definition of data objects and supports some method for establishing relationships among the objects. Therefore, reengineering one database schema into another requires an understanding of existing objects and their relationships.
The following steps define the existing data model as a precursor to reengineering a new database model:

1. Build an initial object model.
2. Determine candidate keys (the attributes are examined to determine whether they are used to point to another record or table; those that serve as pointers become candidate keys).
3. Refine the tentative classes.
4. Define generalizations.

Steps of Software Reverse Engineering:

1. **Collection Information:** This step focuses on collecting all possible information (i.e., source design documents, etc.) about the software.
2. **Examining the Information**: The information collected in step-1 is studied so as to get familiar with the system.
3. **Extracting the Structure**: This step concerns identifying program structure in the form of a structure chart where each node corresponds to some routine.
4. **Recording the Functionality:** During this step processing details of each module of the structure, charts are recorded using structured language like decision table, etc.
5. **Recording Data Flow**: From the information extracted in step-3 and step-4, a set of data flow diagrams is derived to show the flow of data among the processes.

6. **Recording Control Flow:** The high-level control structure of the software is recorded.

7. **Review Extracted Design**: The design document extracted is reviewed several times to ensure consistency and correctness. It also ensures that the design represents the program.

8. **Generate Documentation**: Finally, in this step, the complete documentation including SRS, design document, history, overview, etc. is recorded for future use.

Reverse Engineering Tools:

Reverse engineering tools accept source code as input and produce a variety of structural, procedural, data, and behavioral design. Reverse engineering if done manually would consume a lot of time and human labor and hence must be supported by automated tools. Some of the tools are given below:

1. **CIAO and CIA:** A graphical navigator for software and web repositories and a collection of Reverse Engineering tools.

2. **Rigi:** A visual software understanding tool.

3. **Bunch:** A software clustering/modularization tool.

4. **GEN++:** An application generator to support the development of analysis tools for the C++ language.

5. **PBS:** Software Bookshelf tools for extracting and visualizing the architecture of programs.